

EX NO. 1

SETTING UP HYPERLEDGER FABRIC DEVELOPMENT ENVIRONMENT

AIM

To install and configure the Hyperledger Fabric development environment on a Linux system using Docker, Go, and Fabric binaries, create a blockchain test network, and establish a communication channel between peer organizations.

SOFTWARE REQUIREMENTS

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Go (Version 1.20 or later)
- Git
- Curl
- Hyperledger Fabric Samples
- Hyperledger Fabric Docker Images

HARDWARE REQUIREMENTS

- **Processor:** Intel Core i3 or above
- **RAM:** Minimum 8 GB
- **Storage:** Minimum 20 GB free disk space
- **Internet**

Connection PROCEDURE

Step 1: Update the System

Update the package repositories and upgrade the installed packages.

```
sudo apt update  
sudo apt upgrade -y
```

Step 2: Install Required Packages

Install Git, Curl, jq, and Docker

Compose.

```
sudo apt install -y git curl jq
```

docker-compose Verify installation:

git --version

curl --version

docker-compose --version

```
vboxuser@Ubuntu:~$ git --version
git version 2.43.0
vboxuser@Ubuntu:~$ curl --version
curl 8.5.0 (x86_64-pc-linux-gnu) libcurl/8.5.0 OpenSSL/3.0.13 zlib/1.3 brotli/1.1.0 zstd/1.5.5 libidn2/2.3.7 libpsl/0.21.2 (+libidn2/2.3.7) libssh/0.10.6/openssh/zlib nghttp2/1.59.0 librtmp/2.3 OpenLDAP/2.6.10
Release-Date: 2023-12-06, security patched: 8.5.0-2ubuntu10.12
Protocols: dict file ftp ftps gopher gophers http https imap imaps ldap ldaps mqtt pop3 pop3s rtmp rtsp scp sftp smb smbs smtp smtps telnet tftp
Features: alt-svc AsynchDNS brotli GSS-API HSTS HTTP2 HTTPS-proxy IDN IPv6 Kerberos Largefile libz NTLM PSL SPNEGO SSL threadsafe TLS-SRP UnixSockets zstd
```

Step 3: Verify Docker

Installation Check the

installed Docker version.

docker --version

Start and enable Docker:

sudo systemctl start

docker sudo systemctl

enable docker

Add the current user to the

Docker group: sudo usermod -aG

docker \$USER newgrp docker

Verify Docker:

docker run hello-world

```
vboxuser@Ubuntu:~$ sudo apt-cache policy docker-ce docker-compose-plugin docker-compose
docker-ce:
  Installed: (none)
  Candidate: 5:29.7.2-1-ubuntu.24.04-noble
  Version table:
   5:29.7.2-1-ubuntu.24.04-noble 500
     500 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages
   5:29.7.1-1-ubuntu.24.04-noble 500
     500 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages
   5:29.7.0-1-ubuntu.24.04-noble 500
     500 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages
   5:29.6.2-1-ubuntu.24.04-noble 500
     500 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages
   5:29.6.1-1-ubuntu.24.04-noble 500
     500 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages
   5:29.6.0-1-ubuntu.24.04-noble 500
     500 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages
   5:29.5.3-1-ubuntu.24.04-noble 500
     500 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages
   5:29.5.2-1-ubuntu.24.04-noble 500
     500 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages
   5:29.5.1-1-ubuntu.24.04-noble 500
```

```
chinn@ubuntu:~$ docker --version
Docker version 20.10.1, build 20.10.1-ubuntu-20.04.2
chinn@ubuntu:~$ sudo systemctl start docker
chinn@ubuntu:~$ sudo systemctl enable docker
chinn@ubuntu:~$ sudo systemctl status docker
chinn@ubuntu:~$ systemctl status docker
chinn@ubuntu:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
2f55e6f703b8: Pull complete
d5e72642b7f5: Download complete
Digest: sha256:380b3f6e225511fc3877099f2b1d59cc2cbb48090cc4055f2876e285243cc
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (cmd)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/
```

Step 4: Verify Go

Installation

Check the
installed Go version. go
version

Example Output:

go version go1.22.0 linux/amd64

Step 5: Create a Workspace

Create a directory for the Hyperledger

Fabric project. mkdir -p ~/fabric-dev

cd ~/fabric-dev

Verify the

directory:

pwd

Step 6: Download Hyperledger Fabric Samples

Download the Hyperledger Fabric samples, binaries, and

Docker images. curl -sSL https://bit.ly/2ysbOFE | bash -s

Wait until all required files are downloaded successfully.

Step 7: Navigate to the Fabric Samples Directory

cd fabric-samples

```
vboxuser@Ubuntu:~$ go version
Command 'go' not found, but can be installed with:
sudo snap install go           # version 1.26.7, or
sudo apt install golang-go     # version 2:1.21-2
sudo apt install gccgo-go      # version 2:1.21-2
See 'snap info go' for additional versions.
vboxuser@Ubuntu:~$ mkdir -p ~/fabric-dev
vboxuser@Ubuntu:~$ cd ~/fabric-dev/
vboxuser@Ubuntu:~/fabric-dev$ pwd
/home/vboxuser/fabric-dev
vboxuser@Ubuntu:~/fabric-dev$ curl -sSL https://bit.ly/2ysb0FE | bash -s

Clone hyperledger/fabric-samples repo

==> Cloning hyperledger/fabric-samples repo
Cloning into 'fabric-samples'...
remote: Enumerating objects: 15868, done.
remote: Counting objects: 100% (3446/3446), done.
remote: Compressing objects: 100% (631/631), done.
remote: Total 15868 (delta 3821), reused 2815 (delta 2815), pack-reused 12422 (from 2)
Receiving objects: 100% (15868/15868), 24.09 MiB | 4.99 MiB/s, done.
Resolving deltas: 100% (9146/9146), done.
fabric-samples v2.5.16 does not exist, defaulting to main. fabric-samples main branch is intended to work with recent ve
rsions of fabric.

Pull Hyperledger Fabric binaries

==> Downloading version 2.5.16 platform specific fabric binaries
==> Downloading: https://github.com/hyperledger/fabric/releases/download/v2.5.16/hyperledger-fabric-linux-amd64-2.5.16
-tar.gz
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed

hyperledger/fabric-orderer:2.5.16    e322c57311d3    102MB    38.7MB
hyperledger/fabric-orderer:latest    e322c57311d3    102MB    38.7MB
hyperledger/fabric-peer:2.5         896e75b62de9    232MB    66.8MB
hyperledger/fabric-peer:2.5.16      896e75b62de9    232MB    66.8MB
hyperledger/fabric-peer:latest       896e75b62de9    232MB    66.8MB
```

Step 8: Configure Environment Variables

Add the Fabric binaries to the system PATH.

```
echo 'export PATH=$PATH:$HOME/fabric-dev/fabric-samples/bin' >>
```

```
~/.zshrc source ~/.zshrc
```

Step 9: Verify Hyperledger Fabric Installation

Verify the Peer binary:

```
peer version
```

Verify the Orderer binary:

```
orderer version
```

```
hyperledger/fabric-orderer:2.5.16    e322c57311d3    102MB    38.7MB
hyperledger/fabric-orderer:latest    e322c57311d3    102MB    38.7MB
hyperledger/fabric-peer:2.5         896e75b62de9    232MB    66.8MB
hyperledger/fabric-peer:2.5.16      896e75b62de9    232MB    66.8MB
hyperledger/fabric-peer:latest       896e75b62de9    232MB    66.8MB
vboxuser@Ubuntu:~/fabric-dev$ cd fabric-samples
vboxuser@Ubuntu:~/fabric-dev/fabric-samples$ echo 'export PATH=$PATH:$HOME/fabric-dev/fabric-samples/bin' >> ~/.zshrc
bash: /home/vboxuser/.zshrc: No such file or directory
vboxuser@Ubuntu:~/fabric-dev/fabric-samples$ source ~/.zshrc
vboxuser@Ubuntu:~/fabric-dev/fabric-samples$ peer version

From: v2.5.16
Commit 39A: F871cF9
Go version: go1.20.4
OS/Arch: linux/amd64
Chaincode:
Base Docker Label: org.hyperledger.fabric
Docker Namespace: hyperledger

vboxuser@Ubuntu:~/fabric-dev/fabric-samples$
vboxuser@Ubuntu:~/fabric-dev/fabric-samples$
vboxuser@Ubuntu:~/fabric-dev/fabric-samples$ orderer version

From: v2.5.16
Commit 39A: F871cF9
Go version: go1.20.4
OS/Arch: linux/amd64
vboxuser@Ubuntu:~/fabric-dev/fabric-samples$
```

Step 10: Start the Hyperledger Fabric Test Network

Navigate to the test network directory.

```
cd ~/fabric-dev/fabric-samples/test-network
```

```
vboxuser@Ubuntu: ~/fabric-dev/fabric-samples$
vboxuser@Ubuntu: ~/fabric-dev/fabric-samples$
vboxuser@Ubuntu: ~/fabric-dev/fabric-samples$ orderer version
orderer:
Version: v2.5.16
Commit SHA: f871cf9
Go version: go1.26.4
OS/Arch: linux/amd64

vboxuser@Ubuntu: ~/fabric-dev/fabric-samples$ cd ~/fabric-dev/fabric-samples/test-network
vboxuser@Ubuntu: ~/fabric-dev/fabric-samples/test-network$ ./network.sh up
Using docker: 'docker compose'
Starting nodes with CLI timeout of '5' tries and CLI delay of '1' seconds and using database 'leveldb' with crypto from
'cryptogen'
LOCAL VERSION=v2.5.16
DOCKER_IMAGE_VERSION=v2.5.16
/home/vboxuser/fabric-dev/fabric-samples/test-network/./bin/cryptogen
Generating certificates using cryptogen tool
Creating Org1 identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org1.yaml --output=organizations
org1.example.com
+ res=0
Creating Org2 identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org2.yaml --output=organizations
org2.example.com
+ res=0
Creating Orderer Org identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-orderer.yaml --output=organizations
+ res=0
Generating CSF files for Org1 and Org2
Traceback (most recent call last):
File ~/bin/docker-execute, line 22, in module
```

Start the Fabric test network:

```
./network.sh up
```

The command performs the following tasks:

- Generates cryptographic certificates.
- Creates the required Docker containers.
- Starts the Orderer.
- Starts the peer organizations.
- Creates the Fabric network.

Step 11: Verify Running Containers

Display all active Docker

```
Version table:
1.29.2-subuntu1 500
500 http://in.archive.ubuntu.com/ubuntu noble/universe amd64 Packages
vboxuser@ubuntu: ~/fabric-peer/fabric-samples/test-network$ ./network.sh up
Using Docker and Docker Compose
Starting images with CNI Network id 'cni' (first and only delay of '3' seconds and setting database 'leveldb')
[+] up 7/7
✓ Network fabric_test Created
✓ Volume compose_peer0.org2.example.com Created
✓ Volume compose_orderer.example.com Created
✓ Volume compose_peer0.org1.example.com Created
✓ Container peer0.org2.example.com Started
✓ Container peer0.org1.example.com Started
✓ Container orderer.example.com Started
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
73729b8a4938   hyperledger/fabric-peer:latest      "peer node start"       3 seconds ago Up 2 seconds  0.0.0.0:9051->9051/tcp, [::]:9051->9051/tcp, 7051/tcp, 0.0.0.0:9445->9445/tcp, [::]:9445->9445/tcp
3ad79b31826e   hyperledger/fabric-orderer:latest   "orderer"               3 seconds ago Up 2 seconds  0.0.0.0:7050->7050/tcp, [::]:7050->7050/tcp, 0.0.0.0:7053->7053/tcp, [::]:7053->7053/tcp, 0.0.0.0:9443->9443/tcp, [::]:9443->9443/tcp
ce42d655af3a   hyperledger/fabric-peer:latest      "peer node start"       3 seconds ago Up 2 seconds  0.0.0.0:7051->7051/tcp, [::]:7051->7051/tcp, 0.0.0.0:9444->9444/tcp, [::]:9444->9444/tcp
wed8d8ce6bc1   hello-world                          "/hello"                12 minutes ago Exited (0) 12 minutes ago
keem_holbt
```

containers: docker ps

Expected containers include:

- orderer.example.com

- peer0.org1.example.com
- peer0.org2.example.com

```
ce42d655ef3a hyperledger/fabric-peer:latest "peer node start" 3 seconds ago Up 2 seconds 0.0.0.0:7051->7051/tcp, [::]:7051->7051/tcp, 0.0.0.0:9444->9444/tcp, [::]:9444->9444/tcp
peer0.org1.example.com
eed5d8ce6bc1 hello-world ^/hello 12 minutes ago Exited (0) 12 minutes ago
keen_halbt
vboxuser@buntu: /fabric-dev/fabric-samples/test-network$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
NAMES
73729b8a4930 hyperledger/fabric-peer:latest "peer node start" About a minute ago Up About a minute 0.0.0.0:9051->9051/tcp, [::]:9051->9051/tcp, 7051/tcp, 0.0.0.0:9445->9445/tcp, [::]:9445->9445/tcp
peer0.org2.example.com
3ad79b31826e hyperledger/fabric-orderer:latest "orderer" About a minute ago Up About a minute 0.0.0.0:7050->7050/tcp, [::]:7050->7050/tcp, 0.0.0.0:7053->7053/tcp, [::]:7053->7053/tcp, 0.0.0.0:9443->9443/tcp, [::]:9443->9443/tcp
orderer.example.com
ce42d655ef3a hyperledger/fabric-peer:latest "peer node start" About a minute ago Up About a minute 0.0.0.0:7051->7051/tcp, [::]:7051->7051/tcp, 0.0.0.0:9444->9444/tcp, [::]:9444->9444/tcp
peer0.org1.example.com
vboxuser@buntu: /fabric-dev/fabric-samples/test-network$ ./network.sh createChannel -c mychannel
Using docker and docker compose
Creating channel 'mychannel'
If network is not up, starting nodes with --it timeout of '3' tries and --it delay of '1' seconds and using database 'leveldb'.
Bringing up network
LOCAL_VERSION=1.10
NODE000: [INFO] [0] [1]
[+] up 3/3
✓ Container peer0.org1.example.com Running 0.0.0.0
✓ Container peer0.org2.example.com Running 0.0.0.0
✓ Container orderer.example.com Running 0.0.0.0
```

Step 12: Create a Channel

Create an application channel named **mychannel**.

`./network.sh createChannel -c`

`mychannel` This command:

- Generates the required channel configuration.
- Creates the channel.
- Joins both peer organizations to the channel.
- Configures the anchor peers.

Step 13: Verify Successful Channel Creation

Successful messages include:

Channel 'mychannel' created successfully

Successfully submitted proposal to join

channel Anchor peer set for Org1

Anchor peer set for Org2

Step 14: Stop the Network

After completing the experiment, stop the Fabric network.

```
./network.sh down
```

RESULT

The Hyperledger Fabric development environment was successfully installed and configured. The required Fabric binaries and Docker images were installed, the test network was created successfully, the **mychannel** application channel was generated, both peer organizations joined the channel, and the anchor peers were configured successfully. The development environment is now ready for deploying chaincode (smart contracts) and developing blockchain applications.